

Simulação

A new B (CLRS):

Order 5, logo $2T - 1 = 5 \Rightarrow T = 3$

$$T - 1 = 2$$

↳ I merge :

10, 20, 5, 6, 12, 30, 7, 17

| 10 | $\rightarrow$ | 10, 20 | $\rightarrow$ | 5, 10, 20 | $\rightarrow$ | 5, 6, 10, 20 |

$\rightarrow$ | 5, 6, 10, 12, 20 |

Split:

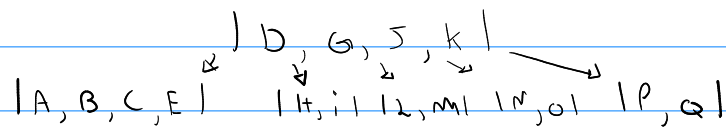
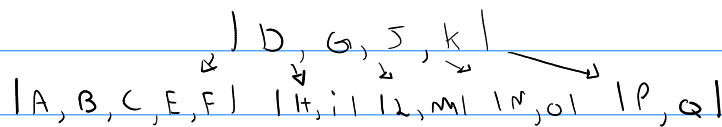
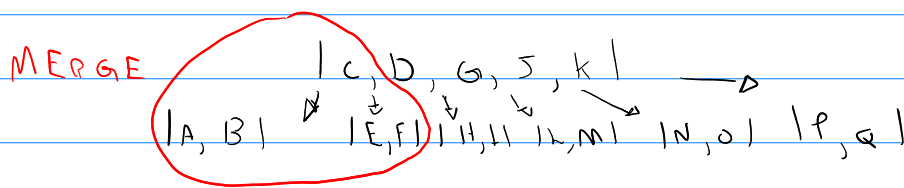
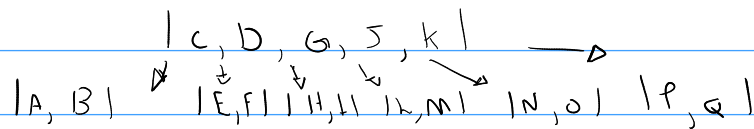
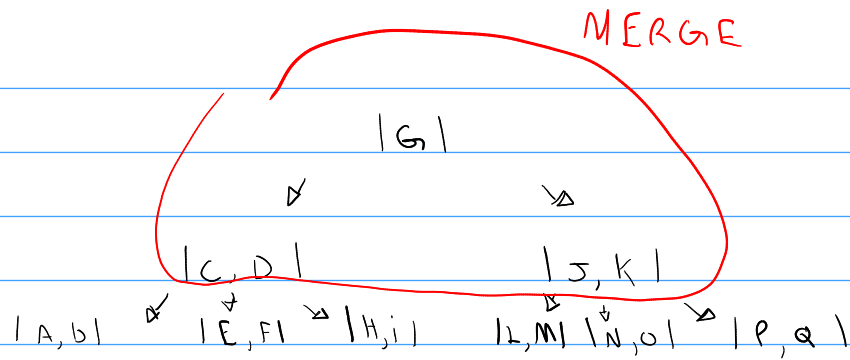
| 10 |
↙ ↘
| 5, 6 | | 12, 20 |

| 10 |
↙ ↘
| 5, 6, 7 | | 12, 17, 20, 30 |

↳ R emov ção :

| 6 |
↙ ↘
| C, D | | 5, K |
↙ ↘ ↙ ↘
| A, B | | E, F | | H, I | | L, M | | N, O | | P, Q |

Sequência de Remoção: F



• **Left-rotate:**

↳ **SPLIT:** nó direito, então a mediana sobe e fica dois filhos a direita

↳ **Remover:** Se vale remover um nó e vejo que ele ficará menor que o mínimo, pago o menor da esquerda ou menor da direita empacotado no lugar do pai e o pai desce.

↳ **Merge:** Se nó da pai empacotado da direita menor esquerda, então.

TRIE :

↳ Inserção :

Lembre-se que :

↳ preenchemos o valor na árvore.

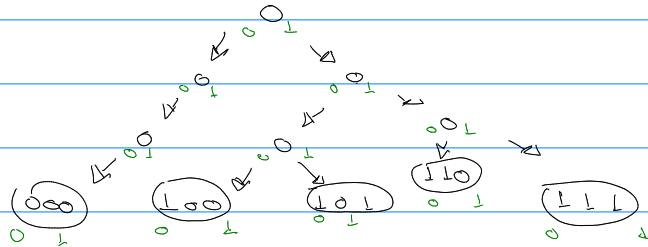
↳ se chegar em NULL, insere

↳ chegou numa folha diferente da desejada.

↳ vai criando nós intermediários até achar algum

dígito em que se diferem.

101, 110, 111, 100, 000



↳ Remoção :

101, não remove o intermediário

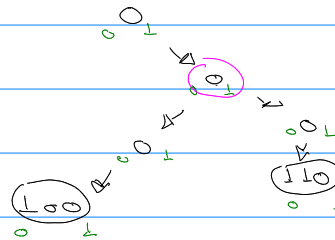
pois o caminho tem outro filho.

111, "

000, remove o intermediário

pois o caminho não tem outro filho.

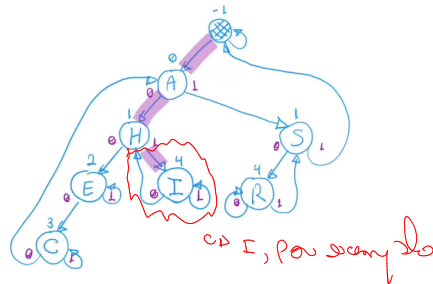
↳ Se 100, " até o



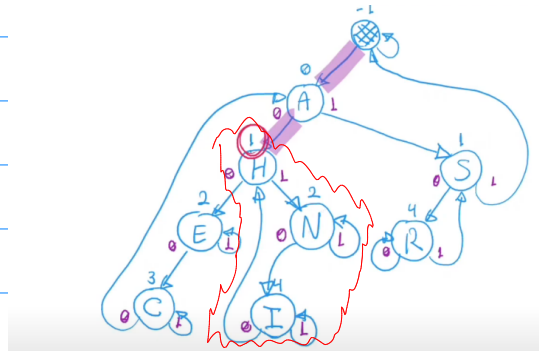
Palavras TRIE:

↳ Imaginemos

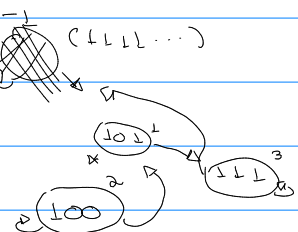
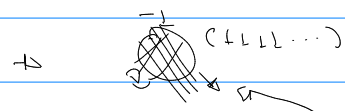
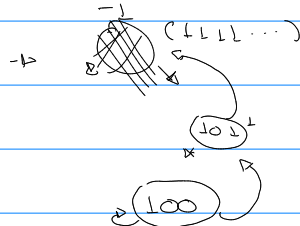
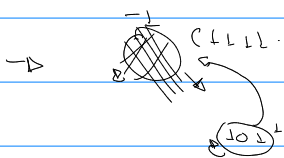
↳ Se não for no meio: vê em qual bit difere e coloca um apontador para ele mesmo e ao anterior.



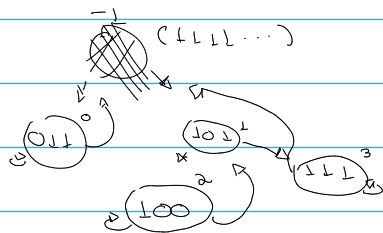
↳ Se for no meio: o chuve pá aponta para a mãe e a mãe para aquela que já existe. Situações como essa ocorrem porque perdemos as verificações de todos os bits para obter exclusivamente para o de dígito. Veja o N, que se difere do H em 02:



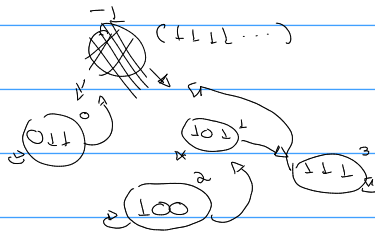
101, 100, 111, 011, 001



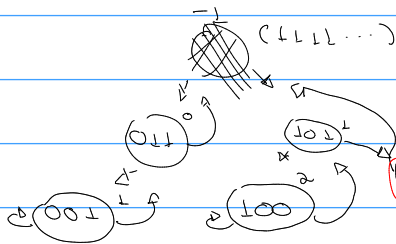
→



→



→



→ Nur müssen implementieren neue Transitionen.